PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Engenharia de Software

VICTOR GABRIEL CRUZ PEREIRA
Matrícula: 861270

DETECÇÃO DE BAD SMELLS E REFATORAÇÃO SEGURA

Análise e validação de um gerador de relatórios JavaScript

Disciplina: Teste de Software

Belo Horizonte
2026

1 ANÁLISE DO CÓDIGO ORIGINAL

Foram analisados o método generateReport e suas regras em src/ReportGenerator.js. A inspeção manual foi complementada pelo ESLint com SonarJS. Os três principais smells e seus impactos são apresentados a seguir.

1.1 Método longo e responsabilidades misturadas

Nas linhas 11-68, o método reúne seleção, prioridade, formatação e soma. Alterar uma regra exige compreender todo o fluxo; testá-la isoladamente requer verificar o relatório completo. A técnica Extract Method separa essas responsabilidades.

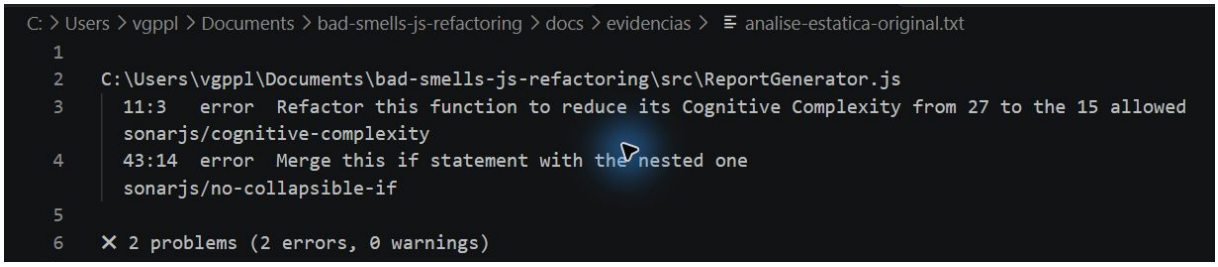
1.2 Condicionais complexas

Nas linhas 27-54, papel do usuário, valor e formato aparecem em decisões aninhadas. Isso dificulta a manutenção e multiplica as combinações de teste. Decompose Conditional e cláusulas de guarda simplificam o fluxo.

1.3 Duplicação de código

As linhas 35-37 e 46-48 repetem a geração CSV. A soma aparece em quatro ramos. Uma mudança pode produzir divergências entre ADMIN e USER, exigindo verificações repetidas. Extrair a linha e centralizar a soma reduz esse risco.

Figura 1 - Resultado do ESLint no código original



Fonte: captura do autor (2026), da saída anterior preservada em analise-estatica-original.txt, aberta no editor. Não é captura histórica do terminal.

O SonarJS mediu complexidade cognitiva 27, acima do limite 15 (11:3), e indicou uma condição unificável (43:14). Acrescentou métrica e precisão aos achados manuais; não reportou funções idênticas. Blocos repetidos não equivalem a funções inteiras iguais.

2 PROCESSO DE REFATORAÇÃO

A prioridade foi decompor generateReport. O original foi preservado e a cópia foi criada em src/ReportGenerator.refactored.js, mantendo o export ReportGenerator, o construtor, a propriedade db e a assinatura pública.

Figura 2 - Trecho original: decisões aninhadas (linhas 43-48)

```
} else if (user.role === 'USER') {
  // Users comuns só veem itens de valor baixo
  if (item.value <= 500) {
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
```

Fonte: código original do projeto (2026). Trecho parcial do ramo USER.

Figura 3 - Trecho refatorado: coordenação do processamento (linhas 18-28)

```
for (const item of items) {
  if (!isVisibleTo(user, item)) {
    continue;
  }

  applyAdminPriority(user, item);
  if (reportType === 'CSV' || reportType === 'HTML') {
    report += createRow(reportType, user, item);
    total += item.value;
  }
}
```

Fonte: elaboração do autor (2026), em ReportGenerator.refactored.js.

Aplicou-se Extract Method para cabeçalho, rodapé, linha, visibilidade e prioridade. O predicado isVisibleTo expressa a seleção, e a cláusula de guarda evita o aninhamento. A geração CSV e a totalização passaram a ter um único ponto de implementação.

Os limites 1000 e 500 foram substituídos por constantes nomeadas, preservando os operadores > e <=. Os auxiliares são internos ao módulo: não foram criadas novas classes ou interfaces públicas.

Foram mantidos a prioridade preexistente, a mutação de item.priority, os formatos desconhecidos e a saída HTML de ADMIN com <tr >. Remover esse espaço alteraria o texto retornado. A suíte foi copiada para __tests__/ReportGenerator.refactored.test.js, mudando apenas o import; a original permanece em tests/.

3 VALIDAÇÃO E CONCLUSÃO

A mesma configuração efetiva foi usada: ESLint 8.57.1, SonarJS 0.21.0, presets recomendados e limite 15 com severidade error. Não houve supressões. O Quadro 1 resume as execuções reais.

Quadro 1 - Comparação das versões

Verificação	**Original**	**Refatorado**
Alertas do ESLint	2 erros; 0 avisos	0 erros; 0 avisos
Complexidade cognitiva	27; limite 15	Sem violação > 15*
Condição unificável	1 ocorrência	Nenhuma
Testes com ESM temporário	4 passam; 1 falha	4 passam; 1 falha

Fonte: evidências do projeto (2026). * O valor exato da cópia não foi emitido; ausência de alerta não significa complexidade zero.

O resultado global foi de 8 aprovações e 2 falhas em 10 testes. A mesma falha de HTML ADMIN já existia na base: espera-se <tr>, mas recebe-se <tr >. O comando npm test sem a opção temporária --experimental-vm-modules falha no carregamento ESM. Portanto, o requisito de 100% de aprovação não foi atingido.

O diff e os hashes confirmaram os originais intactos; o teste copiado difere somente no import. Uma verificação suplementar aprovou 200 cenários, com 400 comparações de saída e estado. Não foram identificadas regressões nesses casos, o que não prova equivalência para todas as entradas possíveis.

Os testes foram uma rede de segurança para comparar o comportamento após cada mudança significativa e distinguir falhas anteriores de regressões. A refatoração tornou as regras mais claras e eliminou os dois alertas na cópia, mantendo explícitas as limitações da validação.

REFERÊNCIAS

PEREIRA, Victor Gabriel Cruz. **Bad-smells-js-refactoring: código e evidências de análise e validação**. 2026. Disponível em: https://github.com/Victorgabrielcruz/bad-smells-js-refactoring. Acesso em: 1 nov. 2026.

PUC MINAS. Sistema Integrado de Bibliotecas. **Orientações para elaboração de projetos de pesquisa, trabalhos acadêmicos, relatórios técnicos e/ou científicos e artigos científicos**. 6. ed. Belo Horizonte: PUC Minas, 2025. Disponível em: https://www.pucminas.br/biblioteca. Acesso em: 1 nov. 2026.